Next-Token Prediction

ES 335 Machine Learning, Semester I 2025–26 Sources: lecture "Next-token prediction", cheat sheet ML19, and the names notebook

What this file is really about:

$$\text{text} \rightarrow (\text{context}, \text{next}) \text{ pairs} \rightarrow \text{embeddings} \rightarrow \text{softmax over the vocabulary} \rightarrow \text{sample, slide, r}$$

How to use this file. Part A builds a small language model of names, step by step. Part B writes it in code. Part C is practice with answers.

Step-up path

You need. Softmax classification and multi-layer perceptrons.

1. Turn text into context and next-character pairs.
2. Count pairs for a baseline.
3. Learn embeddings and concatenate them.
4. Train with cross-entropy.
5. Sample, using a temperature.

You can stop when you can explain how a language model writes one token at a time.

Contents

Part A Maths From the Ground Up

A1 Prediction as classification

Given the last few characters of a name, how likely is each possible next character? Treat this as a classification problem with 27 classes: 26 letters plus one extra token `.` that pads the start and marks the end.

This is the softmax classifier of logistic regression, and a language model does the same thing over a much larger vocabulary.

A2 Pairs from text

Pad the start with `.` and slide a window along. With a context length of 3, the name `abid` gives:

Context	Next
`...`	a
`..a`	b
`.ab`	i
`abi`	d
`bid`	`.`

A name of length L gives $L + 1$ pairs. The last target is `.`, so the model also learns where names stop.

A3 A baseline by counting

Before any neural network, count. A **bigram** model estimates

$$P(\text{next} = c \mid \text{previous} = p) = \frac{\text{count}(p, c) + \alpha}{\sum_{c'} \text{count}(p, c') + 27\alpha}.$$

The smoothing α stops an unseen pair from having probability 0. A longer context fits better, but the number of possible contexts grows as 27^k, and most are never seen. That is why we move to a network, which can generalise between similar contexts.

A4 Embeddings

Give each character a learned vector, stored as a row of a table $E \in \mathbb{R}^{27\times K}$. Looking up row i equals $e_i^\top E$ for the one-hot vector e_i. The table is trained by backpropagation like any other weights.

After training, similar characters end up close together. In the course's trained model, each vowel's nearest neighbour is another vowel, although nothing told it which letters are vowels.

A5 Concatenation keeps the order

Join the context's embeddings into one vector:

$$x = [E_{c_1}, \ldots, E_{c_k}] \in \mathbb{R}^{kK}.$$

A sum or mean would forget which position held which character. With a context of 5 and $K = 4$, the input has 20 numbers. A different context length needs a different input width, so a different network.

A6 The model

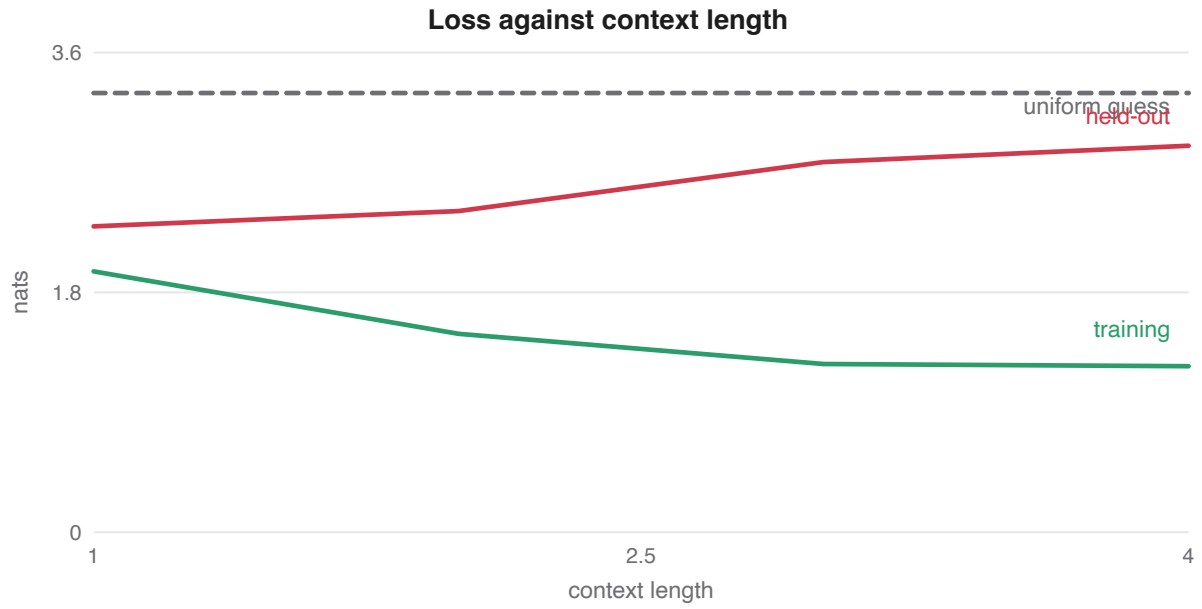
A hidden layer and softmax give

$$h = \sigma(xW_1 + b_1), \qquad z = hW_2 + b_2, \qquad p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

With 64 hidden units the course's model has $108 + 1344 + 1755 = 3207$ parameters.

A7 The loss

Each pair costs $-\ln p_{\text{target}}$, and training minimises the average. A uniform guess costs $\ln 27 = 3.30$ nats, which is the number any model must beat.



Longer context lowers the training loss, but on held-out names the gain stalls as contexts become too rare to count.

A8 Generating text

Start from a context of padding, sample a character from the model's distribution, append it, slide the window, and repeat until `.` appears. A name's probability is the product of its steps.

A9 Temperature

Dividing the logits by a temperature T changes how bold the samples are:

$$p_k(T) = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}} \propto p_k^{1/T}.$$

$T \to 0$ always takes the top character, $T = 1$ is the trained model, and $T \to \infty$ tends to uniform. The ranking of characters never changes, only the gaps between them.

A10 From characters to language models

Large language models use the same recipe. Text is split into tokens, tokens get embeddings, a model scores the next token, and sampling repeats. Transformers replace the fixed context window with attention over all previous tokens, but the task is still next-token prediction.

A11 Why bigram counts are the maximum-likelihood answer

Let θ_{ab} be the probability of b after a, with $\sum_b \theta_{ab} = 1$. The log-likelihood is $\sum_{ab} n_{ab} \log \theta_{ab}$. Adding a Lagrange multiplier for each a and differentiating gives $n_{ab}/\theta_{ab} = \lambda_a$. Normalising,

$$\hat{\theta}_{ab} = \frac{n_{ab}}{\sum_b n_{ab}}.$$

A neural model given only the previous character learns the same probabilities.

A12 Smoothing

A pair never seen gets probability zero, which makes the loss infinite. **Add-one** smoothing adds k to each count: $(n_{ab} + k)/(n_a + kV)$.

A13 Perplexity

$$\text{perplexity} = \exp(\text{mean cross-entropy}).$$

It reads as an effective number of choices. A loss of $\ln 27 = 3.30$ is perplexity 27, a uniform guess over 27 letters.

A14 Beam search

Greedy decoding keeps one sequence. **Beam search** keeps the B highest-scoring prefixes at each step, and expands each by every token. It helps when one correct answer is sought, as in translation. For open-ended writing it tends to be repetitive, so sampling is preferred.

Part B In Code

B1 Building the pairs

```
def pairs(names, block=5):
    X, y = [], []
    for w in names:
        ctx = [0] * block
        for ch in w + '.':
            i = stoi[ch]
            X.append(ctx); y.append(i)
            ctx = ctx[1:] + [i]
    return torch.tensor(X), torch.tensor(y)
```

B2 The model

```
class NextChar(nn.Module):
    def __init__(self, block, vocab, emb, hidden):
        super().__init__()
        self.E = nn.Embedding(vocab, emb)
        self.l1 = nn.Linear(block * emb, hidden)
        self.l2 = nn.Linear(hidden, vocab)

    def forward(self, x):
        h = torch.sin(self.l1(self.E(x).flatten(1)))
        return self.l2(h)
```

B3 Sampling

```
logits = model(ctx) / T
ix = torch.multinomial(F.softmax(logits, -1), 1).item()
```

The notebook **Generating names** in this section trains this model on thousands of Indian first names.

B4 A complete bigram and window model

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
words = ['anna', 'bella', 'carla', 'diana', 'elena', 'flora', 'gina', 'hanna', 'irina', 'ju
chars = ['.'] + sorted(set(''.join(words)))
stoi = {c: i for i, c in enumerate(chars)}
V = len(chars)

# Counting model.
N = torch.zeros(V, V)
for w in words:
    s = '.' + w + '.'
    for a, b in zip(s, s[1:]):
        N[stoi[a], stoi[b]] += 1
P = (N + 0.1) / (N + 0.1).sum(1, keepdim=True)

# Neural model with a window of three characters.
block = 3
X, Y = [], []
for w in words:
    ctx = [0] * block
    for c in w + '.':
        X.append(ctx); Y.append(stoi[c]); ctx = ctx[1:] + [stoi[c]]
X, Y = torch.tensor(X), torch.tensor(Y)

class Window(nn.Module):
    def __init__(self, d=10, h=64):
        super().__init__()
        self.e = nn.Embedding(V, d)
        self.net = nn.Sequential(nn.Linear(block * d, h), nn.Tanh(), nn.Linear(h, V))
    def forward(self, x):
```

```
        return self.net(self.e(x).flatten(1))

m = Window(); opt = torch.optim.Adam(m.parameters(), lr=1e-2)
for i in range(400):
    loss = F.cross_entropy(m(X), Y)
    opt.zero_grad(); loss.backward(); opt.step()

bigram = -torch.log(P[torch.tensor([stoi[w[i - 1]] if i else 0 for w in words for i in rang
                      torch.tensor([stoi[c] for w in words for c in w])]).mean().item()
print('bigram loss', round(bigram, 3), 'window loss', round(loss.item(), 3))
assert loss.item() < 0.5
```

B5 Beam search

```
import torch

def beam(step, start, width=3, length=4):
    beams = [([start], 0.0)]
    for _ in range(length):
        grown = []
        for seq, score in beams:
            log_p = step(seq[-1])
            top = torch.topk(log_p, width)
            grown += [(seq + [i], score + v) for v, i in zip(top.values.tolist(), top.indic
        beams = sorted(grown, key=lambda b: -b[1])[:width]
    return beams

T = torch.log(torch.tensor([[0.1, 0.6, 0.3], [0.8, 0.1, 0.1], [0.2, 0.2, 0.6]]))
out = beam(lambda i: T[i], 0)
print(out[0])
assert out[0][0][:3] == [0, 1, 0]
```

Part C Practice

P1. How many (context, next) pairs does a name of length 6 give?

7.

P2. What loss does a uniform guess over 27 characters have?

$\ln 27 = 3.30$ nats.

P3. A context of 3 characters with embedding size 10. How wide is the input?

30.

P4. With $T = 2$, probabilities 0.64 and 0.36 become what?

Proportional to 0.8 and 0.6, so 0.571 and 0.429.

P5. Why not simply count contexts of length 10?

There are 27^{10} possible contexts and nearly all would never be seen, so counts cannot generalise.

P6. A model predicts the next character with probability 0.1. What is the loss for that pair?

$-\ln 0.1 = 2.30$.

P7. Loss 2.2. Find the perplexity.

$e^{2.2} = 9.03$.

P8. Bigram count $n_{ab} = 3$, total after a is 9, $V = 27$, $k = 1$. Find the smoothed probability.

$4/36 = 0.111$.